

Grounding DINO 开放词汇目标检测与视觉定位复现

Student Name: 王扬皓, 张泰玮, 郑翔越
Student ID: 12310802, 12310801, 12310805

Student Name: 王扬皓, 张泰玮, 郑翔越
Student ID: 12310802, 12310801, 12310805

1 1. Introduction

开放词汇目标检测的目标是使用任意文本描述来定位图像中的目标，而不是只在固定类别集合中做检测。视觉定位是一个相关任务，模型需要根据自然语言表达式定位对应的图像区域，例如“左侧的红色汽车”或“拿着雨伞的人”。这两个任务比传统闭集识别更难，因为模型需要同时具备视觉识别能力和语言理解能力。

本项目复现并评估 Grounding DINO 在开放词汇目标检测（OVD）和视觉定位（VG）两个任务上的表现。我们使用官方 Grounding DINO Swin-T OGC checkpoint，并在不进行 COCO 或 RefCOCO 微调的 zero-shot 设置下评估模型。项目目标是搭建可复现的评测 pipeline，与原论文结果进行比较，并分析复现结果与论文之间的差异来源。

项目主要包括三个部分：方法复现、数据集评测和实验分析。OVD 部分在 COCO val2017 上评估；VG 部分在 RefCOCO、RefCOCO+ 和 RefCOCOg 上评估。最终自实现 OVD pipeline 在 COCO val2017 上达到 **46.16 mAP**，官方 Grounding DINO COCO 脚本达到 **48.50 mAP**。VG 复现结果在五个评测 split 上达到或略高于论文的 zero-shot @1 结果。

2 2. Related works

DETR 将目标检测建模为端到端 transformer 预测问题。DINO 在 DETR 基础上引入 denoising anchor boxes 和更强的 query 初始化策略，提升了检测效果。Grounding DINO 进一步基于 DINO 框架，将检测模型扩展为语言条件下的开放集检测模型。

Grounding DINO 结合 Swin Transformer 视觉主干、BERT 文本编码器和跨模态融合模块。它不只预测固定类别，而是使用文本 prompt 指导检测，因此可以同时用于开放词汇检测和自然语言指代表达式定位。

相关的开放词汇或视觉语言检测方法还包括 GLIP、OWL-ViT、YOLO-World 和 Detic。本项目选择 Grounding DINO，是因为它直接支持 OVD 和 VG 两个任务，并且官方 checkpoint 和评测脚本公开可用。

本项目使用的数据集也是该方向的标准 benchmark。COCO 是常用的目标检测数据集，官方使用 COCO API / pycocotools 计算 mAP。RefCOCO、RefCOCO+ 和 RefCOCOg 用于评估基于自然语言 referring expression 的视觉定位能力。

3 3. Method

本项目复现已有方法，而不是提出新模型。复现模型为 Grounding DINO，checkpoint 为官方 Swin-T OGC 权重 `groundingdino_swint_ogc.pth`。

模型 pipeline 如下：

1. 使用 Swin-T visual backbone 编码图像。
2. 使用 BERT 编码文本 prompt 或 referring expression。
3. 使用 Grounding DINO 跨模态 transformer 融合视觉和文本特征。
4. 解码候选框和 token-aligned phrase scores。
5. 将模型输出转换为对应任务的预测结果。

对于 OVD，输入文本是包含 80 个 COCO 类别名称的 prompt。我们使用 `concat_token prompt mode`，将类别名拼接后送入模型，并将 token span 映射回 COCO category ID。最终预测结果使用 COCO API 评测。

对于 VG，输入文本是 referring expression。模型输出多个候选框后，我们使用 semantic matching strategy 选择得分最高的候选框。如果预测框与 ground-truth box 的 IoU 不低于 0.5，则该样本视为正确。

实现支持两个模型后端：

Backend	Role	Final usage
gdino	官方 GroundingDINO 源码和 OGC checkpoint	主结果后端
hf	HuggingFace IDEA-Research/grounding-dino-tiny	fallback 和早期验证

主要实现文件包括：

- `src/model_wrapper.py`: 统一模型加载和推理接口。
- `src/ovd/eval_coco.py`: COCO OVD 评测。
- `scripts/run_vg_eval.py`: RefCOCO 系列 VG 评测。
- `src/ovd/official_postprocess.py`: OVD token-span 映射。
- `src/vg/box_selection.py`: VG box selection 策略。

4 4. Experiments

4.1 4.1 Datasets

我们在一个目标检测数据集和三个视觉定位数据集上评估。

Task	Dataset	Images	Annotation	Scale
OVD	COCO val2017	<code>data/coco/val2017</code>	<code>instances_val2017.json</code>	5000 images
VG	RefCOCO	COCO train2014	RefCOCO annotations / HF parquet	val, testB

Task	Dataset	Images	Annotation	Scale
VG	RefCOCO+	COCO train2014	RefCOCO+ annotations / HF parquet	val, testB
VG	RefCOCOg	COCO train2014	RefCOCOg annotations / HF parquet	val

本项目使用 zero-shot inference。我们没有在 RefCOCO 上训练或微调模型。因此，VG 结果应当与 Grounding DINO 论文中的 zero-shot @1 结果比较，而不是与 fine-tuned RefCOCO 结果比较。

4.2 4.2 Implementation Details

主要实现使用 PyTorch 和官方 GroundingDINO 源码后端。模型 checkpoint 为 weights/groundingdino_swint_ogc.pth。

关键设置如下：

Item	Setting
Model	Grounding DINO Swin-T OGC
Framework	PyTorch
OVD prompt mode	concat_token
OVD prompt template	{name}
OVD best threshold	box=0.15, text=0.05
VG selection	semantic
VG threshold	box=0.05, text=0.05

OVD threshold 通过 threshold sweep 选择。VG threshold 通过 500 样本验证子集 sweep 选择，其中 box=0.05、text=0.05 得到最佳 Acc@0.5。

我们还验证了 GroundingDINO CUDA extension。在 RTX 5090 服务器上，原始 extension 在 PyTorch 2.8 下无法编译，原因是使用了较旧的 PyTorch C++ API，例如 `tensor.type()` 和 `tensor.data<T>()`。将这些调用 patch 为 PyTorch 2.x 兼容 API 后，`groundingdino._C` 可以成功 import。CUDA extension 提升了推理速度，但没有显著改变 OVD mAP，因此最终精度分析主要关注评测协议和后处理差异。

4.3 4.3 Metrics

OVD 使用 COCO API / pycocotools 计算 COCO mAP。主指标是在 IoU 0.50 到 0.95 范围上平均的 mAP，同时报告 AP50、AP75、小/中/大目标 AP 和 average recall。

VG 使用 Acc@0.5。若所选预测框与 ground-truth box 的 IoU 不低于 0.5，则该预测为正确。该指标对应论文中的 zero-shot @1。

4.4 4.4 Experimental design & results

4.4.1 OVD on COCO val2017

主要 OVD 结果来自 results/exp_2026-05-23_ovd_aligned/metrics.json。

Metric	Result
mAP	46.16
AP50	61.20
AP75	50.39
AP_s	31.26
AP_m	49.08
AP_l	60.72
AR@1	35.88
AR@10	58.39
AR@100	61.88

与论文和官方脚本的比较如下：

Method	COCO val2017 mAP	Notes
Grounding DINO paper	48.4	论文 zero-shot 结果
Official test_ap_on_coco.py	48.50	本项目复现结果
Our self-implemented pipeline	46.16	threshold sweep 后的最佳 full-run 结果

官方脚本结果说明 checkpoint 和数据集准备是正确的。自实现 pipeline 与官方结果之间的差距，主要可能来自后处理、token 到 category 的映射方式，以及评测协议细节。

4.4.2 Visual grounding on RefCOCO-family datasets

主要 VG 结果来自 results/refcoco_gdino/metrics.json。

Dataset / split	Our Acc@0.5	Hits / Total	Paper zero-shot @1	Difference
RefCOCO val	50.85%	5509 / 10834	50.41%	+0.44
RefCOCO testB	45.38%	2312 / 5095	43.21%	+2.17
RefCOCO+ val	51.67%	5559 / 10758	51.40%	+0.27
RefCOCO+ testB	46.45%	2271 / 4889	45.81%	+0.64
RefCOCog val	60.95%	2984 / 4896	60.42%	+0.53

这些结果说明 VG pipeline 与论文 zero-shot 设置基本对齐。五个评测 split 全部达到或略高于论文报告的 zero-shot @1 数值。

4.4.3 Prompt ablation

Prompt ablation 结果来自 results/coco/ablation_prompt.json。

结果如下：

- Prompt template {name}: mAP **47.05**, AP50 **58.80**, 2731 predictions。
- Prompt template a {name}: mAP 17.56, AP50 24.21, 1675 predictions。

- Prompt template a photo of a {name}: mAP 5.24, AP50 6.33, 185 predictions。

直接使用类别名效果最好，因为 COCO OVD 会把 80 个类别名拼接到一个长 prompt 中。更长的模板会消耗 BERT token budget，导致后面的类别被截断或 token 对齐变差。

4.4.4 VG threshold and protocol checks

500 样本子集上的最佳 VG threshold 如下：

box_threshold	text_threshold	Acc@0.5	Samples
0.05	0.05	52.20%	500

我们也比较了两种 VG box selection 策略：

Strategy	Acc@1	Acc@5	Acc@10
semantic	52.20%	89.60%	95.60%
argmax_official	52.60%	89.60%	95.60%

两种策略的 top-1 差距只有 0.4 个百分点，top-5 和 top-10 完全一致。这说明 VG 性能主要受候选框质量影响，而不是由这一具体选择规则决定。

4.4.5 Engineering validation

RTX 5090 CUDA extension 验证结果如下：

Experiment	mAP	Speed observation	Role
5090 fallback	44.60	About 8.8 images/s	工程验证
5090 CUDA fixed	44.60	About 10.7 images/s	工程验证

CUDA extension 可以提升速度，但不能解释 OVD 精度差距。因此，46.16 与 48.50 之间的主要差异更可能来自后处理和评测协议不完全一致。

4.4.6 Limitations

自实现 OVD pipeline 仍低于官方脚本。由于官方脚本使用同一 checkpoint 和同一数据集可以达到 48.50 mAP，因此差距大概率不是模型权重或数据准备导致的。

小目标检测仍然较难：AP_s 为 31.26，而 AP_m 为 49.08，AP_l 为 60.72。这符合预期，因为小目标在下采样后视觉特征更弱，也更难与文本 query 精确对齐。

对于 VG，本项目只评估 zero-shot inference，没有复现论文中 81-89% 范围的 fine-tuned RefCOCO 结果。要复现这些结果，需要在 RefCOCO split 上进行监督训练，并需要额外计算资源。

如果继续推进项目，下一步可以完全对齐官方 COCO 后处理逻辑，在 LVIS 或 ODinW 上评估更稀有类别，并实现 RefCOCO fine-tuning。

5 5. Conclusion

本项目使用 Grounding DINO 完成开放词汇目标检测和视觉定位复现。我们复现了官方模型 pipeline，在 COCO val2017 和 RefCOCO 系列数据集上评测，并分析了 prompt 设计、threshold、selection protocol、CUDA extension 行为和失败模式。

自实现 OVD pipeline 在 COCO val2017 上达到 **46.16 mAP**，官方 COCO 评测脚本达到 **48.50 mAP**。VG pipeline 在五个 RefCOCO 系列 split 上分别达到 **50.85%**、**45.38%**、**51.67%**、**46.45%** 和 **60.95%** Acc@0.5，达到或略高于论文 zero-shot 结果。

总体而言，项目完成了预期的复现和评测目标。主要局限是自实现 OVD 后处理尚未完全对齐官方评测脚本。不过，实验已经清楚地定位了这一差距，并说明模型、数据集和核心评测 pipeline 都是有效的。

6 Reference

1. Shilong Liu et al. "Grounding DINO: Marrying DINO with Grounded Pre-Training for Open-Set Object Detection." ECCV 2024.
2. Hao Zhang et al. "DINO: DETR with Improved DeNoising Anchor Boxes for End-to-End Object Detection." ICLR 2023.
3. Tsung-Yi Lin et al. "Microsoft COCO: Common Objects in Context." ECCV 2014.
4. Licheng Yu et al. "Modeling Context in Referring Expressions." ECCV 2016.
5. COCO API: <https://github.com/cocodataset/cocoapi>
6. pycocotools: <https://github.com/ppwwyyxx/cocoapi>
7. GroundingDINO official repository: <https://github.com/IDEA-Research/GroundingDINO>
8. GLIP repository: <https://github.com/microsoft/GLIP>
9. OWL-ViT documentation: https://huggingface.co/docs/transformers/model_doc/owlvit
10. YOLO-World repository: <https://github.com/AILab-CVC/YOLO-World>
11. Detic repository: <https://github.com/facebookresearch/Detic>

7 Contributions

王扬皓 (12310802): 33.33%

张泰玮 (12310801): 33.33%

郑翔越 (12310805): 33.33%